

Learning goals: Basics of shell scripting continued: `exit`, `if-then-fi` construct/statement with string comparisons, exercises with standard streams

NOTE: Please do all your work in a new folder called `lab11`. Save the script for each exercise separately as `question1.sh`, `question2.sh` and so on. At the end of the lab, please 'push' your scripts to GitHub!

EXERCISES

- (1) The Linux system handles every object as a file. This includes the input and output process. Each file object is identified using a *file descriptor*, which is a non-negative integer that uniquely identifies open files in a session. The bash reserves the first three file descriptors (0,1 and 2) for special purposes. The file descriptor 0 stands for STDIN (standard input), 1 for STDOUT (standard output) and 2 for STDERR (standard error). The workings of these are shown below. Try these commands first to understand how these work.
 - (i) Understanding STDOUT and STDERR

```
ls
ls > listoffiles
ls 1> listoffiles
```

The symbol `>` automatically redirects any non-error output of the screen to `listoffiles`. The `1 >` means that the standard output in particular to the `listoffiles`. Now try the following command where the directory called `newdir` does not exist.

```
ls -l . newdir
ls -l . newdir 1>presentfiles 2>filesnotpresent
```

Explain what happened in the `newdir` example. Did you notice that the successful part of the `ls` command went to `presentfiles` and the error part went to `filesnotpresent` file?
 - (ii) What is the result of `ls -l . newdir >listoffiles` ? Explain how this command worked.
- (2) What is the difference between `-e,-s` and `-f` in the use of `if-block`? Demonstrate with an example.
- (3) Write a shell script to check if a string is empty or not (`-n` operator or `-z` operator in the `if` statement)
- (4) String comparisons are tricky when the `>` or `<` operations are involved. Let us look at this example below.

```
val1=Jayashree
val2=Nagesh
if [ $val1 > $val2 ]; then
    echo "$val1 is greater than $val2"
else
    echo "$val1 is lesser than $val2"
fi
```

The above script should execute just fine. However, list the contents of your directory now, and you should find a new file called `Nagesh`! This is not what we wanted although the comparison worked successfully. The shell basically interpreted that it should create a file called `Nagesh` due to the presence of the `>` symbol, hence the unwanted result. Delete the file `Nagesh`. To avoid this, use `$val1 \> $val2` in the `if` statement and run it again. What is the result this time? Explain what happened and why.

- (5) String comparisons with upper and lower cases are also tricky. The `if` handles upper and lower cases in way that is opposite to the `sort` command. Do the following and explain what happened:
- (1) Define variables `var1` with value "Testing" and `var2` with value 'testing'
 - (2) Write in a shell script, an if statement involving `>` comparison of the two strings. Remember to use the `\` with the operator as in the last question. Use appropriate `echo` statements
 - (3) Execute the shell script and note down the result of which variable is greater than the other.
 - (4) Save the values of the two variables in a new file called 'teststringfile'.
 - (5) Execute the `sort` command as `sort teststringfile` and note down the results. How do they compare to the result from the `if` statement? Are they the same? (Hint: They should not be) Why is this the case?